Code Explanation and Presentation Preparation

Study only
Keep this document outside your submission ZIP.

This guide explains the supplied phishing classification project in execution order. Use it to run the code, trace the evaluation decisions and defend the results in your own words. The assessment rubric rewards preparedness, structure, depth and delivery; reading a script without understanding the experiment will not establish those qualities.

How to use the package

01_Submission contains the editable report and professional presentation. Complete the cover, recording link and group details before submitting. 02_Code_and_Evidence contains the runnable project and results to retain for Assessment 4. 03_Study_Only_Do_Not_Submit contains this walkthrough, rubric mapping and rehearsal material. You can omit that entire study folder from your submission while retaining a personal copy.

File	Responsibility
Start_Mac.command	Finds Python and starts local setup and Jupyter
run_project.py	Creates the environment; checks dependencies; launches actions
train.py	Loads data, splits, tunes, evaluates and exports evidence
Phishing_Lab.ipynb	Displays and explains results; optionally reruns training
requirements.txt	Lists required packages and version constraints
data and results	Source data and the evidence produced by the experiment

Reading order

Start with the Mac instructions, then study data loading and splitting. Next understand pipelines and tuning, followed by the test metrics and feature experiment. Finish with the rubric checklist and timed rehearsal. Keep the code open beside the explanation so that you can locate each operation.

Core explanation to remember

The model predicts a recorded label from thirty categorical website indicators. Identical input patterns are grouped to prevent overlap across evaluation partitions. Cross-validation selects the forest before the test scores are inspected. Its strong F1 still leaves missed threats and false alarms.

1 Running the project on your Mac

Extract the complete package. In Terminal, type cd followed by a space, drag the phishing_project folder from 02_Code_and_Evidence into Terminal, and press Return. That folder contains train.py and Start_Mac.command. Then run:

```
bash Start_Mac.command
```

The launcher accepts Python 3.11, 3.12 or 3.13. It looks for an existing local environment first, otherwise a supported Python installation. It sets numerical-library thread limits, changes to the project directory and calls setup. First setup requires internet access to install dependencies. If supported Python is absent, install Python 3.12 from python.org and reopen Terminal.

What setup does

run_project.py setup creates .venv using the selected Python interpreter. It installs requirements.txt through that environment's Python, registers a kernel within the environment and runs a dependency and dataset check. It does not replace your system Python. The launcher then starts JupyterLab using .venv/bin/python.

```
.venv/bin/python run_project.py check
.venv/bin/python run_project.py notebook
.venv/bin/python run_project.py train
```

Use check to diagnose dependencies, notebook to open the notebook, and train to regenerate all experiment outputs. Select the PRT565 Phishing (.venv) kernel in Jupyter or the project .venv interpreter in VS Code. The notebook starts with RUN_TRAINING = False and reads the bundled results. Change it to True only when you want a new training run. Copy results/ first because a rerun overwrites it.

Diagnosing a failure

If imports fail, confirm the selected interpreter is .venv/bin/python and rerun the setup command. If Python cannot be found, install a supported version. If the notebook cannot import train, open the complete project folder instead of an isolated notebook. run_project.py writes startup_error.txt when a Python action fails; share that file and the final Terminal error lines for a specific diagnosis.

The bundled experiment and saved predictions have been checked in the available environment. Your Mac's GUI, dependency installation and original failure have not been observed directly. This package therefore does not certify that a particular Mac error has been resolved.

2 Loading data and defining the target

```
ROOT = Path(__file__).resolve().parent
records, _ = arff.loadarff(p)
df = pd.DataFrame(records).astype(int)
X = df.drop(columns='Result')
y = (df.Result == -1).astype(int)
```

ROOT anchors paths to train.py, so the working directory does not decide where data and results are located. load_data first uses data/source/Training Dataset.arff. If it is absent, it extracts that named member from the included original ZIP; only if the ZIP is also absent does it attempt a download. The unused archive member is not the training source.

SciPy parses the ARFF records. pandas creates a table and converts the category strings into integers. X has shape (11055, 30), while y has one binary target per row. Removing Result is essential: leaving the answer among the inputs would make the evaluation meaningless.

The expression df.Result == -1 is True for a phishing record and False for a legitimate one. Converting the Boolean series to int yields phishing = 1 and legitimate = 0. This mapping determines the meaning of every subsequent binary metric.

```
groups = pd.util.hash_pandas_object(X, index=False).to_numpy()
assert len(np.unique(groups)) == len(X.drop_duplicates())
```

Hashing X without its index gives identical input rows the same group value, even when their labels differ. The count assertion checks that distinct input patterns have not collapsed into fewer hash groups. Grouping excludes y deliberately: grouping by the entire row would allow identical inputs with opposing labels to cross the split.

Audit checks

The loader asserts the expected 11,055 by 31 source shape, legal input codes and target values. main records missing values, duplicate counts, class counts, split sizes and a SHA-256 checksum. These checks catch unexpected source changes; they do not establish that the original labels are correct. The recorded file has no missing inputs, 5,785 unique feature patterns and 64 patterns with conflicting labels.

Explain it aloud

“These are already-extracted categories. A URL length code is not the actual length. I remove the label from X, make phishing the positive class and group exact input patterns before splitting.”

3 Splitting without exact pattern overlap

```
outer = StratifiedGroupKFold(
    n_splits=5, shuffle=True, random_state=565)
train, test = next(outer.split(X, y, groups))
assert not set(groups[train]) & set(groups[test])
```

The outer splitter creates five possible folds. next takes only the first, by a fixed rule. Its test indices select 2,210 rows and its development indices select 8,845. This is one holdout; it is not an average of five test evaluations. The intersection assertion verifies that no exact input group appears in both partitions.

```
inner = StratifiedGroupKFold(
    n_splits=3, shuffle=True, random_state=566)
folds = list(inner.split(
    X.iloc[train], y.iloc[train], groups[train]))
```

The inner indices are relative to the development subset. GridSearchCV later receives X.iloc[train] and y.iloc[train], so these positions align. It would be wrong to pass these local positions directly as if they indexed the original full dataset. The code separately asserts group separation within every development fold.

What validation means

Within one CV iteration, the model fits on two development folds and is scored on the remaining fold. Repeating this three times evaluates each development observation in validation once. Every model receives the same stored fold list, making the partition comparison consistent. Stratification tries to balance class proportions but cannot split a group to achieve perfect proportions.

Why the test remains separate

Changing settings after looking at test performance turns the test into another validation set. The code finishes the searches and selects the model family before calculating test predictions. Keep the same discipline when making extensions: propose changes using development evidence and reserve new independent data for a new final assessment.

Limits

Grouping exact inputs cannot prove that training and test sites have different domains or dates, because those identifiers are unavailable. Retaining duplicate rows also means frequently observed patterns count multiple times in the final metrics. These limitations belong in the presentation, even though the overlap assertion passes.

4 Encoding and model pipelines

```
OneHotEncoder(categories=[[-1, 0, 1]] * 30,
              handle_unknown='error', sparse_output=False)
```

Each original feature receives three reserved indicator columns. With category order [-1, 0, 1], a value of -1 becomes [1, 0, 0], 0 becomes [0, 1, 0], and 1 becomes [0, 0, 1]. Thirty features therefore yield ninety inputs. Some indicators remain constant when a feature uses only two categories. The code does not infer category vocabulary from held-out labels or statistics.

```
def category_codes(X):
  return np.asarray(X, dtype=int) + 1
```

CategoricalNB needs nonnegative category indices. The deterministic shift maps -1 to 0, 0 to 1 and 1 to 2. min_categories=3 reserves all three possibilities per feature. This shift is a relabelling, not normalization or a new measurement. Naive Bayes uses categorical likelihoods rather than numerical distances.

Pipeline

A Pipeline combines encoding and a model under one fit/predict interface. When GridSearchCV clones and fits a candidate on a training fold, the pipeline's preprocessing step is fitted there too. The same transformation is then applied during validation and prediction. This is especially important for the supervised feature selector used later.

Class	Role and important setting
DecisionTreeClassifier	Single tree; tune max_depth and min_samples_leaf
RandomForestClassifier	150 trees; randomized ensemble; n_jobs=1
CategoricalNB	Category likelihoods; tune smoothing alpha
MLPClassifier	Hidden layers (32, 16); tune L2 regularization alpha

The ANN receives ninety one-hot inputs, learns two hidden representations and produces a binary-class score. ReLU is its hidden activation and Adam updates the weights. max_iter=250 caps training iterations; it is not a promise that every fit takes exactly 250 iterations. early_stopping=False avoids a separate ungrouped internal validation holdout. Warnings are recorded so that incomplete convergence is not silently hidden.

5 Tuning and selecting the primary model

```
search = GridSearchCV(
   pipe, grid, scoring='f1', cv=folds, n_jobs=1,
   refit=True, return_train_score=True, error_score='raise')
search.fit(X.iloc[train], y.iloc[train])
fitted[name] = search.best_estimator_
```

pipe is the candidate pipeline, and grid contains its permitted parameter values. Names such as model__alpha mean the alpha parameter of the pipeline step named model. The double underscore routes a setting to a nested object. scoring="f1" uses phishing as the positive class because y is coded 0 and 1.

The tree and forest each have four configurations, Naive Bayes has three, and the ANN has two. Each configuration is fitted across three folds. refit=True trains the best configuration again on all 8,845 development rows. best_estimator_ is that refitted pipeline, while best_score_ is its selected configuration's mean validation F1. The best estimator is not trained on the test set.

```
winner = max(cvrows, key=lambda r: r['cv_f1'])['model']
```

The largest CV F1 selects Random Forest at 94.58%, before test outcomes are accessed. This is the declared selection rule, not a conclusion chosen after seeing 96.38% test accuracy. return_train_score=True also exposes generalization gaps; the forest's mean training F1 is 98.81%, above its validation F1.

Why small search grids

A manageable grid makes the work reproducible and practical to explain. It also limits the conclusion: a different ANN architecture or wider search might change the ranking. Different families have different budgets, so do not call the comparison exhaustive or equal-compute.

Warnings and runtime

warnings.catch_warnings records warnings from the four primary searches. threadpool_limits and n_jobs=1 limit parallel numerical work. Search times describe the machine used for that run; they are not estimates for your Mac. error_score="raise" stops on an invalid fit rather than treating it as a valid candidate. The recorded primary searches produced an empty warning log.

6 Selecting ten original features

```
def discrete_mi(X, y):
   return mutual_info_classif(
      X, y, discrete_features=True, random_state=565)
subset = Pipeline([
   ('select', SelectKBest(discrete_mi, k=10)), *base.steps])
```

Mutual information measures how much knowing a feature reduces uncertainty about the label. discrete_features=True matches these categorical inputs. SelectKBest retains the ten features with the highest training-set scores. It acts on original columns before one-hot expansion, so ten features here means thirty reserved encoded columns, not ten binary indicators.

base is a clone of the winning full-model pipeline. Cloning keeps configuration but discards fitted state. If its encoder is OneHotEncoder, the code changes its category list to ten features before placing it after the selector. A twenty-feature mismatch would otherwise produce a shape error.

```
subset_cv = GridSearchCV(
   subset, {}, scoring='f1', cv=folds, n_jobs=1,
   error_score='raise')
subset_cv.fit(X.iloc[train], y.iloc[train])
```

The empty grid contains one fixed configuration. Each training fold fits its own selector using only that fold’s labels; the validation fold does not choose its own features. The final refit uses the full development set and may select a different set from an individual fold. get_support retrieves the final selected column positions, which are saved in selection.json.

Interpret the comparison honestly

The family and hyperparameters were selected using the same development CV in the earlier full-feature experiment. The subset CV score is therefore an exploratory comparison, not a completely nested unbiased estimate. The fixed test comparison shows F1 decreasing from 95.91% to 93.50%, a 2.40 percentage-point drop using unrounded values. False alarms increase from 38 to 78 and misses from 42 to 51.

Do not interpret mutual information as a causal explanation or a ranked account of why the forest made one prediction. No extraction-cost benchmark was performed. You can say that fewer inputs were tested; you cannot claim a measured runtime or financial saving.

7 Predictions and metrics explained

```
pred = model.predict(X.iloc[test])
prob = model.predict_proba(X.iloc[test])[:, 1]
tn, fp, fn, tp = confusion_matrix(
   y.iloc[test], pred, labels=[0, 1]).ravel()
```

predict returns the chosen class, while predict_proba returns one column per class. With classes [0, 1], column 1 is the score assigned to phishing. These scores support ROC AUC. They should not automatically be interpreted as well-calibrated probabilities for present-day websites.

labels=[0, 1] fixes the confusion-matrix order. Rows represent actual labels and columns represent predictions. Flattening row by row therefore yields TN, FP, FN and TP. Reversing label order without changing this unpacking would reverse the interpretation.

Metric	Forest calculation	Meaning
Accuracy	(1193 + 937) / 2210 = 96.38%	Share of all predictions correct
Precision	937 / 975 = 96.10%	Share of alerts that are phishing
Recall	937 / 979 = 95.71%	Share of phishing detected
F1	1874 / 1954 = 95.91%	Harmonic balance of precision and recall
ROC AUC	0.9931 from scores	Ranking across decision thresholds

The forest produces 937 true positives, 1,193 true negatives, 38 false positives and 42 false negatives. Recall’s denominator contains all actual phishing cases, including misses. Precision’s denominator contains all alerts, including false alarms. Keep those questions distinct during your explanation.

The majority baseline

DummyClassifier(strategy="most_frequent") always predicts legitimate because that is the larger development class. It attains 55.70% test accuracy while missing all 979 phishing records. Since it predicts no phishing, its phishing precision denominator is zero; zero_division=0 records zero rather than raising an undefined-metric warning.

Why not tune a threshold now

Reducing an alert threshold might improve recall and increase false alarms, but choosing it from these test outcomes would compromise the evaluation. Threshold experiments belong on development data with a separately preserved final test.

8 Evidence files and notebook behaviour

Output	What it lets you verify
audit.json	Counts, label mapping and source checksum
split_manifest.csv	Each original row ID, group and split
feature_codes.csv	Observed categories for every input
*_cv.csv and cv_summary.csv	Candidate scores, variability and selected settings
metrics.csv and classification_reports.json	Aggregated scores and class-level evaluation
predictions.csv	Each test row's actual label, prediction and score
selection.json	CV-selected family and final ten features
environment.json and warnings.json	Run versions and recorded primary-search warnings
demo_examples.json	One saved example for each available outcome type

The prediction table contains 13,260 rows: six methods multiplied by 2,210 test observations. The code assembles the table, writes it to predictions.tmp and replaces predictions.csv. This avoids exposing a partially written prediction CSV during that write. It does not make the whole results directory an atomic transaction; other outputs can already have changed if a later step fails.

The notebook finds the project folder before importing train, displays the audit and results, and optionally runs main. The default False training flag makes it suitable for inspecting bundled evidence. A displayed score is not evidence of a fresh run unless you explicitly trained and checked the regenerated output files.

Saved examples

demo_examples.json includes a correctly detected phishing case, a missed phishing case and a false alarm for the selected model when available. The code takes the first matching test row in each category. These are illustrative examples, not a random sample. Their raw feature values do not prove the causal reason for a model decision. Do not invent a live URL: the supplied ARFF does not provide it.

Model persistence

The script keeps fitted estimators in memory and exports results, not serialized models. Reopening the notebook with training disabled does not load a trained scanner. A new prediction service would require additional model persistence and feature-extraction work outside the current project.

9 Presenting for the actual rubric

Criterion	Marks	Preparation action
Preparedness	6.5	Run the notebook, locate evidence and practise questions
Structure	6.5	Follow problem, data, methods, results and conclusion
Depth	7	Explain grouping, model choices, errors and limitations
Delivery	10	Rehearse audible speech, clear pace and confident transitions

Source: Assessment 3_PRT565_2026.pdf, page 3. Structure and Depth are marked as applicable at group level. The rubric supplies these segments and maxima; it does not provide detailed HD band descriptors. This guide translates them into preparation actions, not a guarantee of a grade.

Timing and group participation

The presentation contains eleven main slides and four backup slides. The main-slide note budgets total 320 seconds, leaving 40 seconds within one member's six-minute maximum. If you have a group, allocate speaking parts to every actual member; keep each member at or below six minutes and the complete presentation at or below fifteen minutes. The limit is a maximum, not a target. Adjust the allocation after rehearsal rather than assuming every speaker needs six minutes.

Practical rehearsal

First explain the project without slides in one minute. Next record a complete timed take while displaying the deck. Listen for quiet audio, rushed technical terms, long pauses and sentences copied directly from the slides. Practise saying what each metric means before giving its number. Pause briefly when changing from methods to results. Use backup slides only when needed; do not read every detail aloud.

For depth, defend three decisions: grouping repeated inputs, selecting by CV F1 and retaining a separate test set. Explain one limitation immediately after the main result. For preparedness, know where metrics.csv and split_manifest.csv are located and how to distinguish saved results from a new run. For structure, connect each section to the research question rather than listing software features.

Before recording

Replace missing group details in the deck and report. Display the final slide file, close distracting windows and check microphone volume. After recording, test the sharing link without signing into your own account. The submitted report's second page must contain that accessible link.

10 Questions you should answer independently

Why not use the Telco dataset again

The lecturer requires a different dataset for Assessment 3. This phishing dataset is then retained for the linked Assessment 4.

Why is phishing class 1

It makes the positive-class metrics consistently measure threat detection.

Why retain repeated and conflicting records

They are part of the source evidence. Grouping prevents exact-input overlap without deleting difficult cases to improve the score.

Why choose the forest

It had the highest mean development CV F1 before test evaluation. Its test score confirmed strong performance on this partition.

Why use Categorical rather than Gaussian Naive Bayes

The inputs are discrete category codes; categorical likelihoods match that representation.

Why did the ANN not win

Only this architecture and limited search were evaluated. The experiment does not establish a universal reason or show that neural networks are generally worse.

What does 95.91% F1 hide

The forest still misses 42 phishing records and raises 38 false alarms. Operational costs and current traffic may differ.

Is the subset result unbiased nested CV

No. Feature selection is fitted within folds, but model family and settings came from the earlier searches on those same development folds.

Can this scan a website pasted into a form

No. It classifies pre-extracted indicators. Live URL collection and feature extraction are not implemented.

What will carry into Assessment 4

The same dataset and auditable experiment files. The final report structure must follow the Assessment 4 brief once available.

11 References and final self check

Mohammad, R., & McCluskey, L. (2012). *Phishing websites [Data set].* UCI Machine Learning Repository. https://doi.org/10.24432/C51W2X

Scikit-learn developers. (n.d.-a). *Common pitfalls and recommended practices.* Scikit-learn. https://scikit-learn.org/stable/common_pitfalls.html

Scikit-learn developers. (n.d.-b). *MLPClassifier.* Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Scikit-learn developers. (n.d.-c). *Naive Bayes.* Scikit-learn. https://scikit-learn.org/stable/modules/naive_bayes.html

Scikit-learn developers. (n.d.-d). *StratifiedGroupKFold.* Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

Independent explanation check

Before submission, explain the target mapping, grouped split, CV selection rule and confusion matrix without reading the guide. Open train.py and point to the operations responsible. Then explain one error example and one limitation. If you cannot yet explain a section, rehearse it rather than adding more jargon or more algorithms.

Submission check

Complete every bracketed personal field, remove unused member rows and add the recording link. Confirm the deck identifies the real group members. Follow your unit's requirements for acknowledging assistance. Preserve the code and original evidence for Assessment 4. Exclude this study folder from the submission ZIP; retain your own copy for revision.